RIPHAH INTERNATIONAL UNIVERSITY

SDA-Pro

Security Incident Response & Threat Mitigation Platform

Submitted By

Adeel Abbas, hamza Ahsan, usama Bin Ahmed
SAP ID: 56157, 56187,
Section: SE (6-1)

Submitted To

Sir Uzair Rasheed

SEMESTER PROJECT

Introduction

SDA-Pro (Security Incident Response and Threat Mitigation Platform) is a security management system that helps organizations detect, analyze, and respond to cyber security threats. The system collects alerts from different sources, processes them, creates incidents, and automatically performs response actions.

The main purpose of this project is to reduce the time required to identify and handle security incidents. The system follows Software Design and Architecture principles and implements multiple design patterns to make the software scalable, maintainable, and reusable.

Objectives

- Detect security alerts from multiple sources.
- Normalize alerts into a common format.
- Enrich alerts with additional information.
- Manage the complete incident lifecycle.
- Perform automated response actions.
- Provide real-time dashboard updates.
- Maintain audit logs for compliance.
- Demonstrate all required design patterns and architectures.

Project Scope

The system supports:

- Alert Ingestion
- Alert Enrichment
- Incident Management
- Threat Intelligence Integration
- Response Orchestration
- Dashboard Monitoring
- Audit Logging

The system does not include:

- Machine Learning Detection
- Mobile Application
- Billing System
- Multi-Tenant Deployment

Functional Requirements

1. Alert Ingestion

The system receives alerts from:

- Firewall Logs
- EDR Endpoints
- Cloud SIEM
- Threat Intelligence Feeds

2. Alert Enrichment

The system enriches alerts using:

- Geo Location
- Threat Reputation
- Asset Information
- User Information

3. Incident Management

Incident States:

- New
- Under Triage
- Containment
- Eradication
- Recovery
- Closed

4. Response Actions

- Block IP
- Disable User
- Quarantine File
- Isolate Endpoint

- Escalate Incident

5. Dashboard

- Real-Time Monitoring
- Incident Tracking
- Alert Stream
- Action History

6. System Architecture

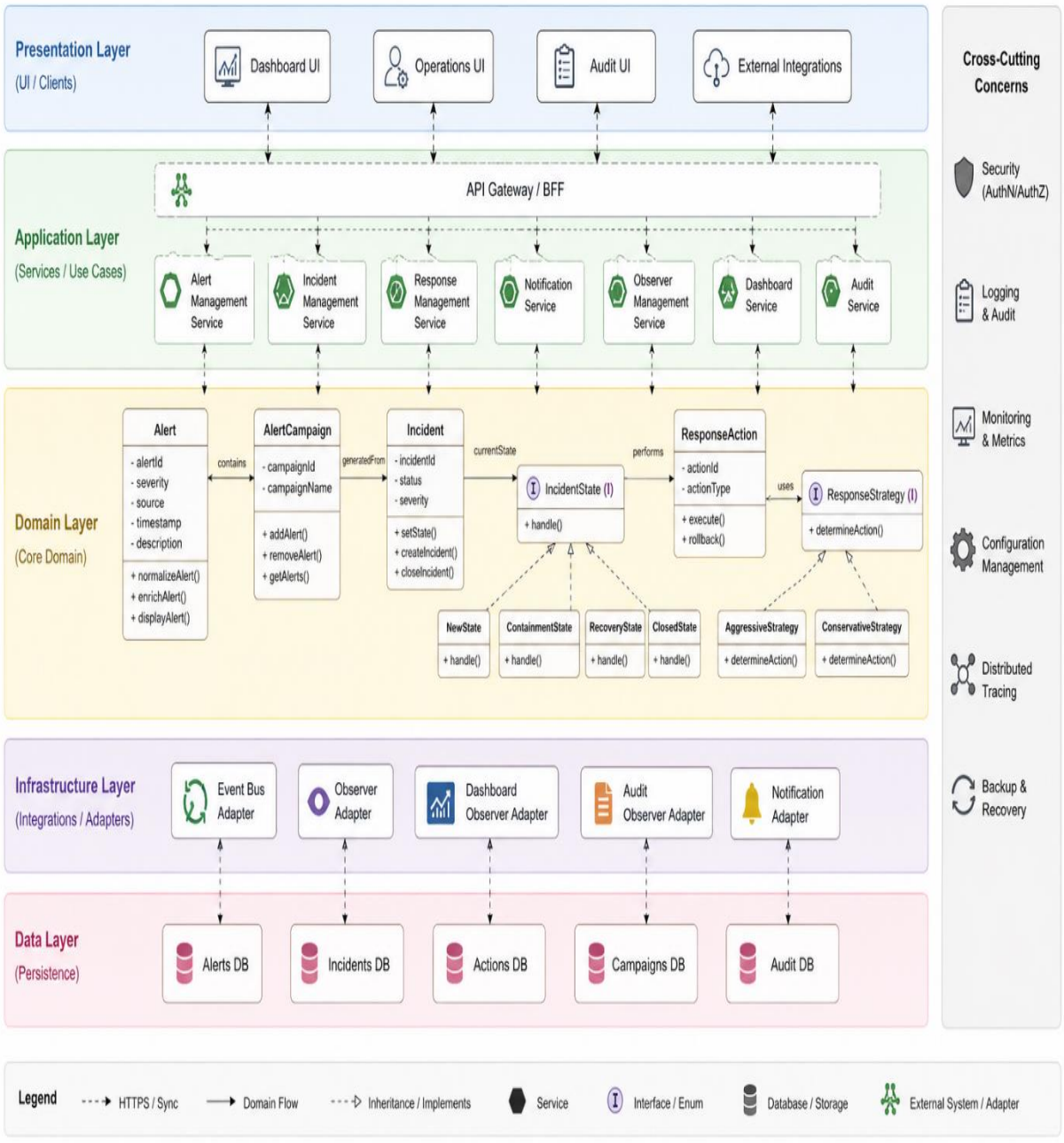
- SOA (Service Oriented Architecture)
- MVC Architecture
- Layered Architecture
- Event Driven Architecture

7. Design Patterns Used

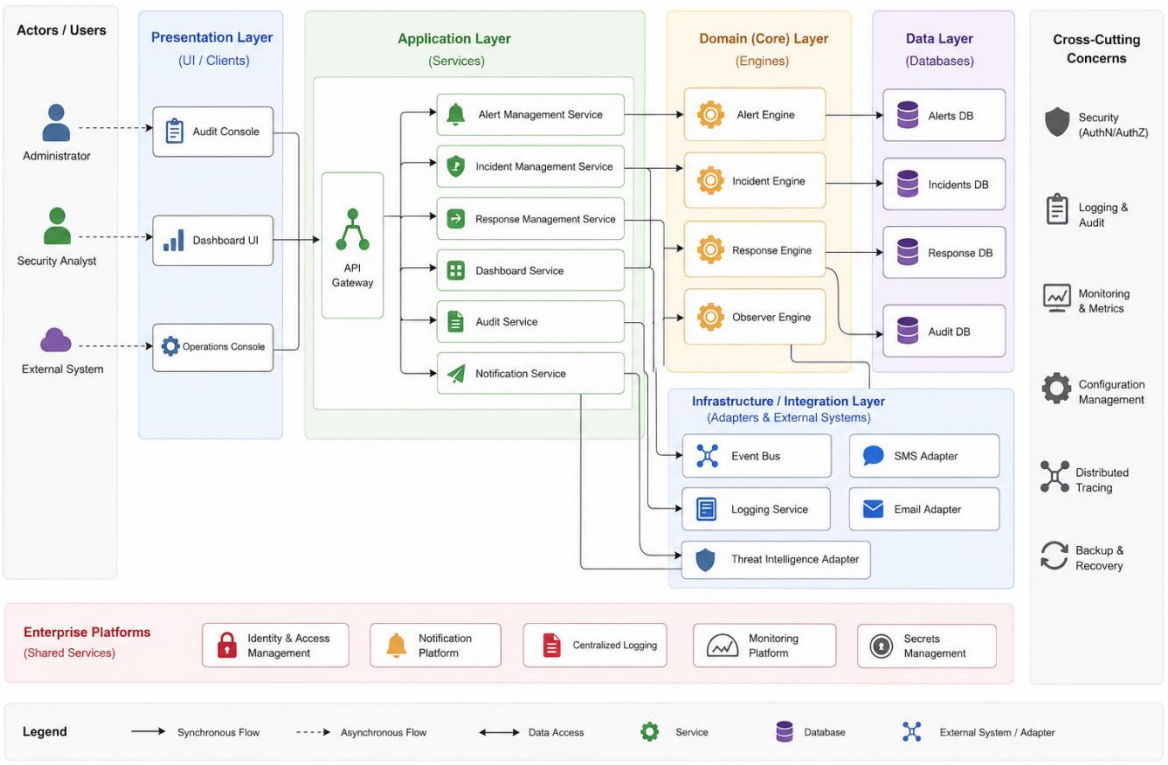
- Singleton
- Factory Method
- Abstract Factory
- Composite
- Facade
- Adapter
- Decorator
- Proxy
- State
- Chain of Responsibility
- Observer
- Strategy

8. UML Diagrams

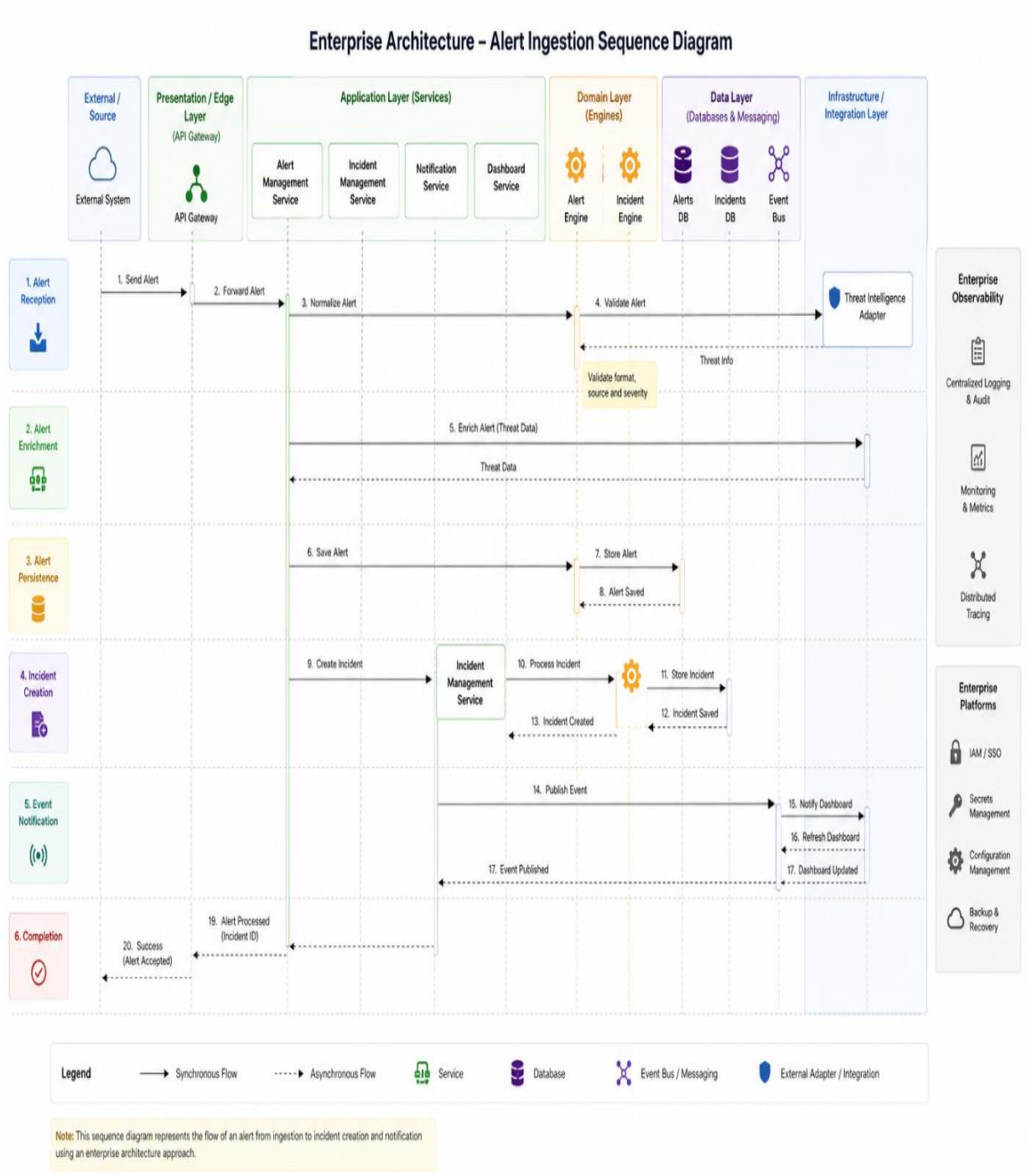
1. UML Class Diagram



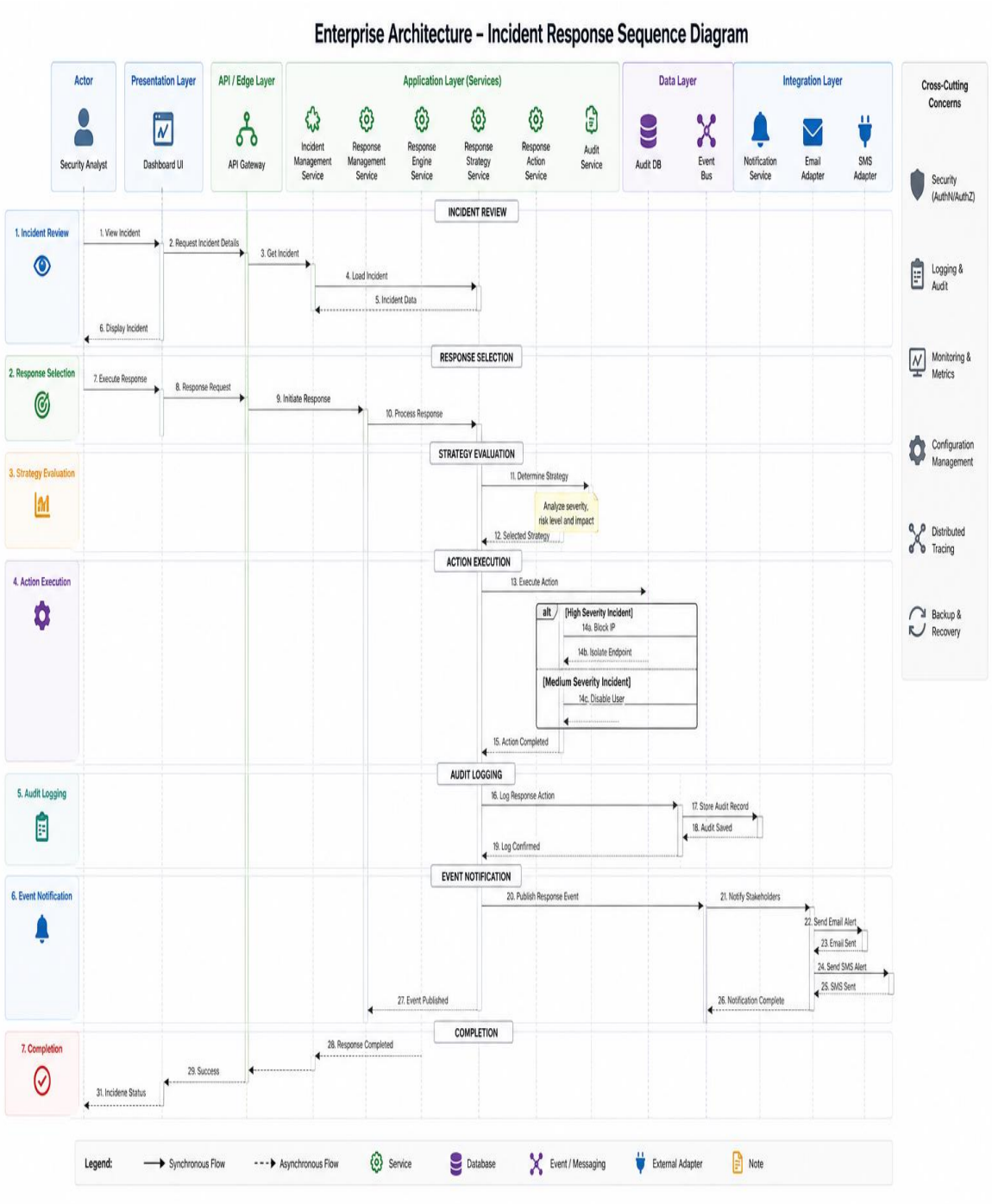
2. UML Component Diagram



3. UML Sequence Diagram (Alert Ingestion)


Enterprise Architecture – Alert Ingestion Sequence Diagram
External / Source
External System
Presentation / Edge Layer
(API Gateway)
API Gateway
Application Layer (Services)
Alert Management Service
Incident Management Service
Notification Service
Dashboard Service
Domain Layer (Engines)
Alert Engine
Incident Engine
Data Layer
(Databases & Messaging)
Alerts DB
Incidents DB
Event Bus
Infrastructure / Integration Layer
Threat Intelligence Adapter
1. Alert Reception
1. Send Alert
2. Forward Alert
3. Normalize Alert
4. Validate Alert
Threat Info
Validate format, source and severity
2. Alert Enrichment
5. Enrich Alert (Threat Data)
Threat Data
3. Alert Persistence
6. Save Alert
7. Store Alert
8. Alert Saved
4. Incident Creation
9. Create Incident
Incident Management Service
10. Process Incident
11. Store Incident
12. Incident Saved
13. Incident Created
5. Event Notification
14. Publish Event
15. Notify Dashboard
16. Refresh Dashboard
17. Dashboard Updated
17. Event Published
6. Completion
19. Alert Processed (Incident ID)
20. Success (Alert Accepted)
Enterprise Observability
Centralized Logging & Audit
Monitoring & Metrics
Distributed Tracing
Enterprise Platforms
IAM / SSO
Secrets Management
Configuration Management
Backup & Recovery
Legend
Synchronous Flow
Asynchronous Flow
Service
Database
Event Bus / Messaging
External Adapter / Integration
Note: This sequence diagram represents the flow of an alert from ingestion to incident creation and notification using an enterprise architecture approach.

4. UML Sequence Diagram (Incident Response)



9. Project Structure

```
src/
|
 ├── main/
|
 ├── alert/
|  ├── Alert.java
|  ├── AlertCampaign.java
|  ├── AlertManager.java
|  └── AlertFactory.java
|
 ├── incident/
|  ├── Incident.java
|  ├── IncidentManager.java
|  ├── IncidentState.java
|  ├── NewState.java
|  ├── ContainmentState.java
|  ├── RecoveryState.java
|  └── ClosedState.java
|
 ├── response/
|  ├── ResponseAction.java
|  ├── ResponseStrategy.java
|  ├── AggressiveStrategy.java
|  ├── ConservativeStrategy.java
|  ├── ResponseManager.java
|  └── IncidentResponseFacade.java
|
 ├── observer/
|  ├── Observer.java
|  ├── DashboardObserver.java
|  ├── AuditObserver.java
|  └── EventBus.java
|
 ├── adapter/
|  ├── ThreatIntelAdapter.java
```

```
|  ├── EmailAdapter.java
|  └── SMSAdapter.java
|
├── decorator/
|  ├── ResponseDecorator.java
|  ├── AuditDecorator.java
|  └── LoggingDecorator.java
|
├── proxy/
|  ├── ThreatIntelProxy.java
|  └── ResponseProxy.java
|
├── chain/
|  ├── Handler.java
|  ├── ValidationHandler.java
|  ├── EnrichmentHandler.java
|  └── ClassificationHandler.java
|
├── database/
|  ├── AlertRepository.java
|  ├── IncidentRepository.java
|  └── AuditRepository.java
|
└── Main.java
```

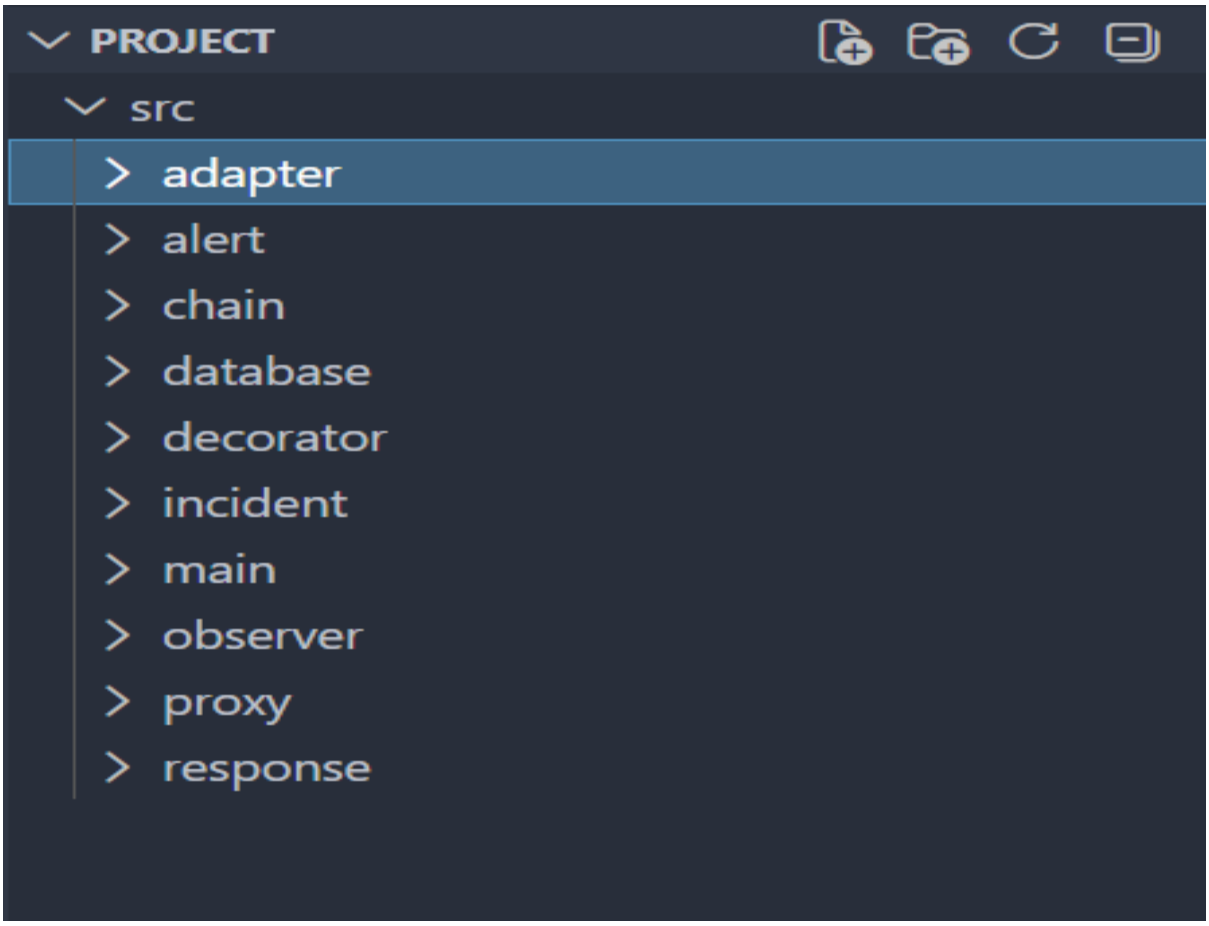
Class Summary

Package	Main Classes
alert	Alert, AlertCampaign, AlertFactory
incident	Incident, IncidentState, NewState, RecoveryState
response	ResponseAction, ResponseStrategy
observer	Observer, DashboardObserver, EventBus
adapter	ThreatIntelAdapter, EmailAdapter
decorator	ResponseDecorator, AuditDecorator
proxy	ThreatIntelProxy

Package	Main Classes
chain	ValidationHandler, EnrichmentHandler
database	AlertRepository, IncidentRepository
main	Main

Design Patterns Mapping

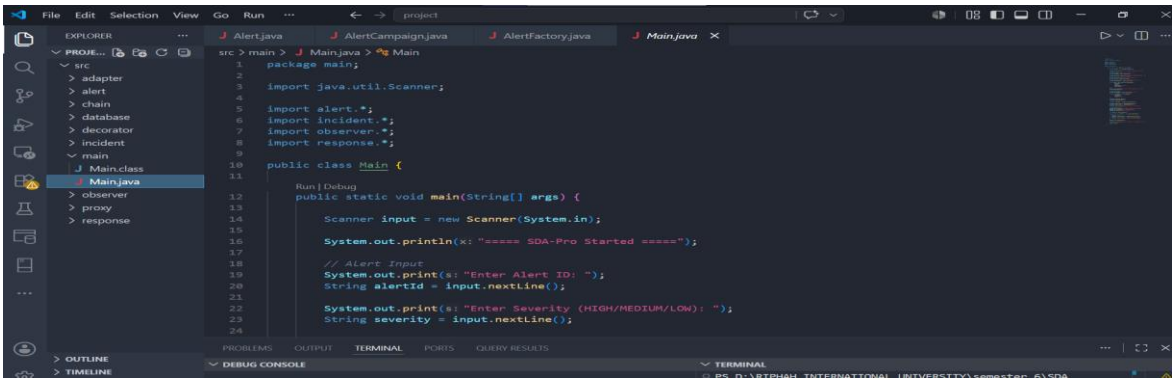
Design Pattern	Class
Singleton	EventBus
Factory Method	AlertFactory
Abstract Factory	ResponseFactory
Composite	AlertCampaign
State	IncidentState
Strategy	ResponseStrategy
Observer	Observer, DashboardObserver
Adapter	ThreatIntelAdapter
Proxy	ThreatIntelProxy
Decorator	ResponseDecorator
Facade	IncidentResponseFacade
Chain of Responsibility	ValidationHandler



10. Java Code Implementation

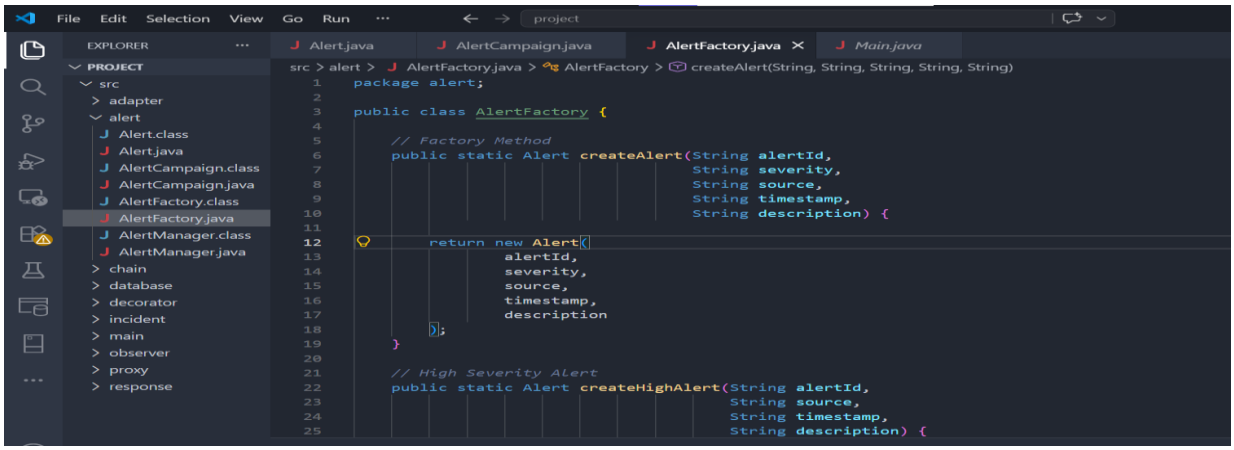
1. Main.java

Purpose:
The Main class serves as the entry point of the SDA-Pro system. It accepts user input, creates alerts, manages incidents, triggers observer notifications, and executes the selected response strategy.



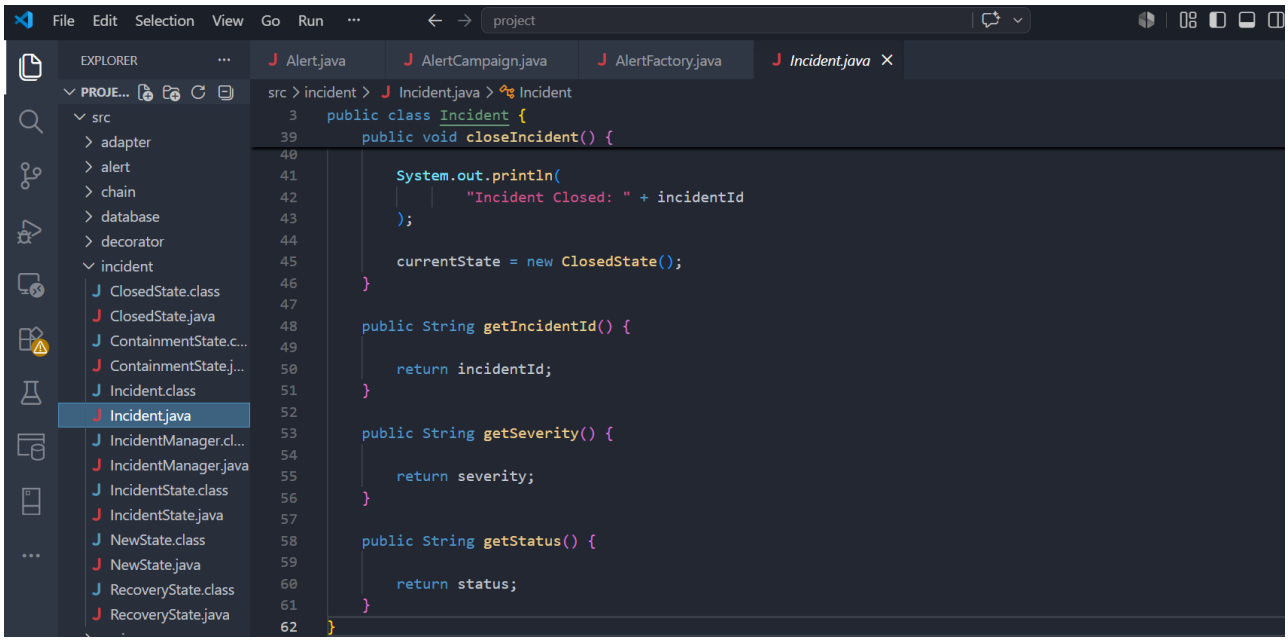
2. AlertFactory.java

Purpose:
The AlertFactory class implements the Factory Method Pattern. It creates different types of alert objects based on the severity level provided by the user (High, Medium, or Low).



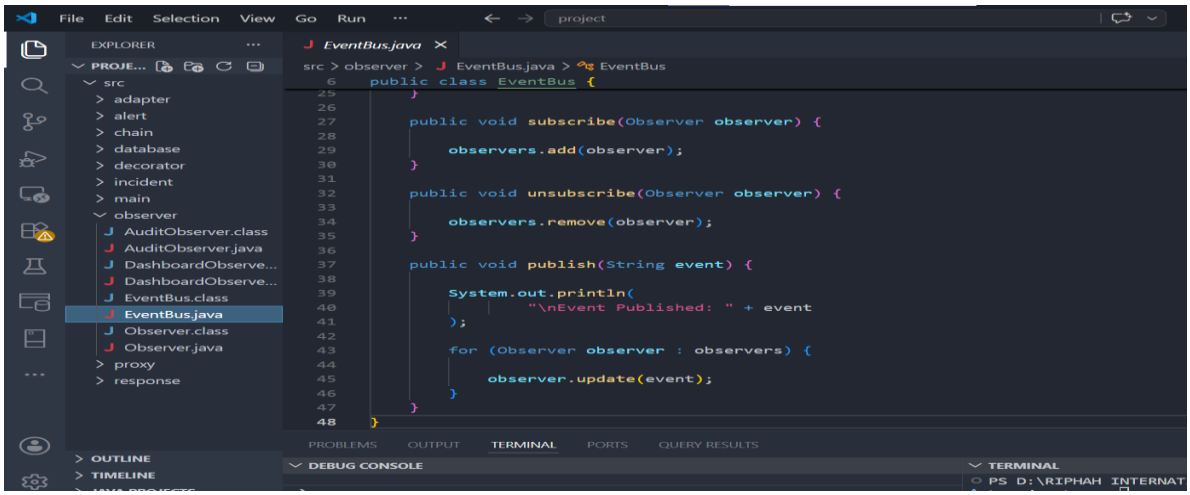
3. Incident.java

Purpose:
The Incident class stores incident-related information such as incident ID, severity, and status. It is responsible for managing the incident lifecycle and handling incident operations.



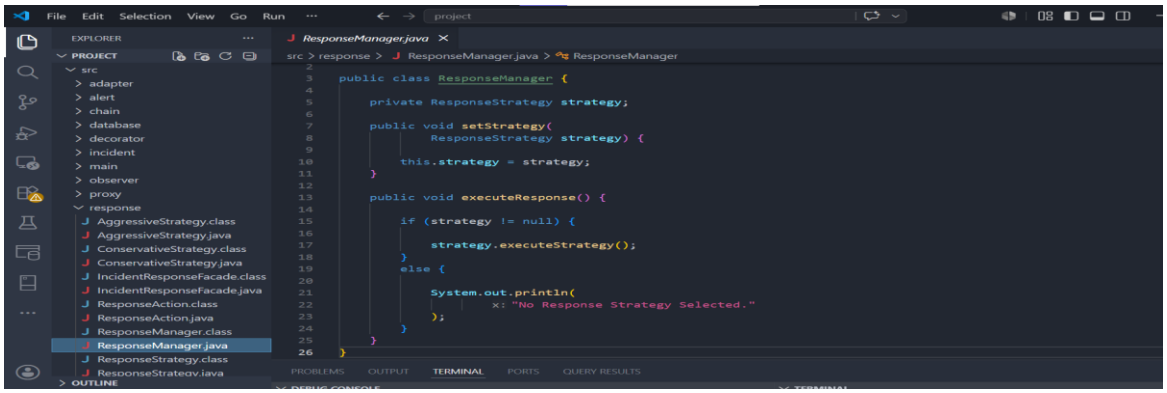
4. EventBus.java

Purpose:
The EventBus class implements the Singleton and Observer Patterns. It manages observer registration and sends notifications to subscribed observers whenever a security event occurs.



5. ResponseManager.java

Purpose:
The ResponseManager class implements the Strategy Pattern. It allows the system to select and execute different response strategies, such as Aggressive or Conservative responses, based on user choice.



11. Output Screenshots

Screenshot 1 – User Input

```
===== SDA-Pro Started =====
Alert ID: A101
Severity: HIGH
Source: Firewall
Timestamp: 02-06-2026
Description: Unauthorized Access
```

Screenshot 2 – Incident Creation

```
Incident Created: INC001
Incident is in NEW state.
```

Screenshot 3--Response Started

```
=== Incident Response Started ===
Executing Aggressive Response Strategy..
Blocking IP Address...
```

Screenshot 4 – Observer Pattern

```
Dashboard Updated: New Security Incident
Audit Log Updated: New Security Incident
```

Screenshot 5 – Strategy Pattern:

```
Executing Aggressive Response Strategy...
Blocking IP Address...
Isolating Endpoint...
=== Incident Response Completed ===
```

12. Conclusion

SDA-Pro (Security Incident Response and Threat Mitigation Platform) successfully demonstrates the practical implementation of Software Design and Architecture concepts. The system provides alert management, incident handling, automated response execution, dashboard monitoring, and audit logging functionalities. Multiple design patterns such as Singleton, Factory Method, Observer,

Strategy, Adapter, Proxy, Decorator, Facade, and Chain of Responsibility have been implemented to improve scalability, maintainability, and reusability. The project achieves its objectives of managing security incidents efficiently while following modern software architecture principles.

THE END
